

When Do You Need to Use Recursion?

You never *need* to use recursion. No programming problem *requires* recursion. This chapter has shown that recursion has no magical power to do things that iterative code in a loop with a stack data structure cannot do. In fact, a recursive function might be an overcomplicated solution for what you're trying to achieve.

However, as the exponent functions we created in the previous section show, recursion can provide new insights into how to think about our programming problem. Three features of a programming problem, when present, make it especially suitable to a recursive approach:

- It involves a tree-like structure.
- It involves backtracking.
- It isn't so deeply recursive as to potentially cause a stack overflow.

A tree has a *self-similar* structure: the branching points look similar to the root of a smaller subtree. Recursion often deals with self-similarity and problems that can be divided into smaller, similar subproblems. The root of the tree is analogous to the first call to a recursive function, the branching points are analogous to recursive cases, and the leaves are analogous to the base cases where no more recursive calls are made.

A maze is also a good example of a problem that has a tree-like structure and requires backtracking. In a maze, the branching points occur wherever you must pick one of many paths to follow. If you reach a dead end, you've encountered the base case. You must then backtrack to a previous branching point to select a different path to follow.

[Figure 2-5](#) shows a maze's path visually morphed to look like a biological tree. Despite the visual difference between the maze paths and the tree-shaped paths, their branching points are related to each other in the same way. Mathematically, these graphs are equivalent.